



FACULTY OF COMPUTING

GROUP PROJECT

SECJ 2013

DATA STRUCTURE AND ALGORITHM

SECTION 11

LECTURER'S NAME: DR. FAZLIATY EDORA BINTI FADZLI

GROUP 10: MELANIE PERKINS

BIL	GROUP MEMBERS	MATRIC NO
1	LOH HUI YI	A23CS0106
2	HAZIQ BIN SHAHMEN	A23CS0082
3	TAI YI TIAN	A23CS0272

Table of Contents

1.0 Problem Statement.....	2
2.0 Data Structure & Algorithms.....	3
3.0 Input/Output.....	5
4.0 Implementation Overview.....	6
System Workflow.....	6
Key Components.....	6
Integration of Components.....	7
Error Handling.....	7
Tools and Environment.....	7
5.0 Testing & Results.....	8
6.0 Conclusion & Reflection.....	13
7.0 References.....	14

1.0 Problem Statement

In this project, we are trying to develop a system to help the hospital staff for managing hospital appointment queues. First of all, we plan to implement a data structure by using Queue that will store patient's records such as patient name and appointment time. The system will also use the Sequent Sort algorithm to sort the patients based on their appointment times. By implementing this system, the hospital staff can give the patients a better service experience by managing the appointment schedule more efficiently and reducing waiting time. Overall, the system will help hospitals reduce the error and confusion by organizing the patient appointment time chronologically.

2.0 Data Structure & Algorithms

Data Structure used: **Queue**

Queue is First-in, First-out (FIFO) property which means that the first item inserted into the queue is the first item to leave the queue. Normally, the basic structure of a queue will have a data structure that holds the queue, a head and a rear.

In our system, we have defined a struct, Patient, to hold the patient's data for the name and appointment time.

```
struct Patient {
    string name;
    string appointmentTime; // Format: HH:MM
    Patient* next; // Pointer for linked list

    // Constructor
    Patient(string n, string time) : name(n), appointmentTime(time), next(nullptr) {}
};
```

We also defined a head and a rear which call tail, for the queue data structure.

```
Patient* loadPatientsFromFile(const string& inputFile) {
    ifstream inFile(inputFile);
    Patient* head = nullptr;
    Patient* tail = nullptr;
```

After that, we will read the data from the input file and insert the patient's data into the queue.

```
string line;
while (getline(inFile, line)) {
    //size_t commaPos = line.find(',');
    auto commaPos = line.find(',');
    //if (commaPos != string::npos) {
        string name = line.substr(0, commaPos);
        string time = line.substr(commaPos + 1);
        Patient* newPatient = new Patient(name, time);

        if (!head) {
            head = tail = newPatient;
        } else {
            tail->next = newPatient;
            tail = newPatient;
        }
    }
}
```

After the system generates an output file that sorts patients' records by appointment time, we will delete the data from the queue (deQueue). The first data inserted into the queue will be delete first.

```
// Function to free memory allocated for the linked list
void freeLinkedList(Patient* head) {
    while (head) {
        Patient* temp = head;
        head = head->next;
        delete temp;
    }
}
```

Algorithm implemented: **Sequential Sort**

Sequential Sort will sort the data from smallest to largest. In our system, the time is recorded in 24-hour time notation, thus our algorithm will compare the time and sort it by using bubble sort.

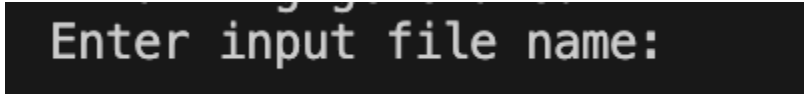
```
// Sequential sort function to sort linked list by appointment time
Patient* sequentialSort(Patient* head) {
    if (!head || !head->next) {
        return head;
    }

    for (Patient* i = head; i->next != nullptr; i = i->next) {
        for (Patient* j = i->next; j != nullptr; j = j->next) {
            if (j->appointmentTime < i->appointmentTime) {
                swap(i->name, j->name);
                swap(i->appointmentTime, j->appointmentTime);
            }
        }
    }
    return head;
}
```

3.0 Input/Output

Input:

For the input, we develop to allow the users to enter the file name that they wish to read to give more flexibility for users, enabling a more user friendly and customizable experience.



```
Enter input file name:
```

Output:

We also allow the user to enter the output file name that they wish to save, thus users can more conveniently manage their file and work.



```
Enter output file name: █
```

4.0 Implementation Overview

The implementation of our system for managing hospital appointment queues integrates the **FIFO concept of queues** and the **Sequential Sort algorithm** with **file operations** to ensure efficient and accurate processing of patient data. This section outlines the workflow and key functionalities of the system.

System Workflow

The implementation of the hospital appointment queue involves the following key steps:

1. **Reading Input Data:** Patient records are loaded from an input file (input.txt) and stored in a queue using a linked list structure.
2. **Sorting Records:** The queue is sorted by appointment time using the Sequential Sort algorithm to ensure chronological order.
3. **Writing Output Data:** The sorted records are saved into an output file (output.txt), ready for use by the hospital staff.

Key Components

- **Queue Structure:** The queue is implemented using a linked list where each node stores a patient's name, appointment time, and a pointer to the next node. This structure ensures dynamic memory management and supports the **First-In, First-Out (FIFO)** principle.
- **File Operations:**
 - **Input Handling:** The program reads patient data from the file, parses it, and dynamically allocates memory to add each patient record to the queue.
 - **Output Handling:** After sorting, the records are written back to a file in the desired format.

- **Sequential Sort:** The program sorts the queue using Sequential Sort, which iterates through the linked list to compare and swap nodes based on their appointment times. Although simple, this approach is effective for small datasets.

Integration of Components

1. **Input File Parsing:** The loadPatientsFromFile() function reads each line of the input file, extracts the patient's name and appointment time, and appends a new node to the linked list.
2. **Sorting Process:** The sequentialSort() function processes the linked list, comparing each node's appointment time with others and arranging them in ascending order.
3. **Output File Writing:** The writePatientsToFile() function iterates through the sorted linked list and writes each patient's details to the output file.

Error Handling

- **Input Errors:** If the input file is missing or cannot be opened, the program displays an error message and exits gracefully.
- **Memory Management:** All dynamically allocated memory for the linked list is freed after processing to prevent memory leaks.

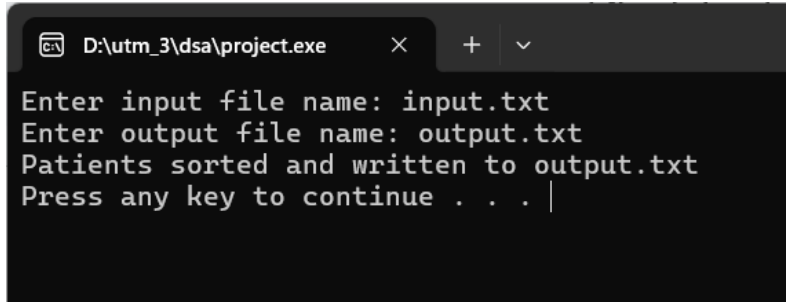
Tools and Environment

- **Programming Language:** C++ for efficient handling of data structures and file I/O.
- **Development Tools:** Code was written, compiled, and tested using an IDE or a text editor with support for C++ (e.g., Visual Studio Code).

5.0 Testing & Results

Testing 1:

The input file consists of a total of 10 patient records.



```
D:\utm_3\dsa\project.exe
Enter input file name: input.txt
Enter output file name: output.txt
Patients sorted and written to output.txt
Press any key to continue . . . |
```



```
input.txt
1 Levi Ackerman,10:30
2 Eren Eager,01:40
3 Hange Zoe,12:45
4 Armin Arlert,16:01
5 Annie Leonhart,04:15
6 Erwin Smith,19:18
7 Zeke Yeager,16:43
8 Sasha Braus,23:59
9 Connie Springer,17:56
10 Jean Kirchtein,03:19
11
```

The output file shows the sorted appointment time for 10 patient records (ascending order)



```
output.txt
1 Eren Eager,01:40
2 Jean Kirchtein,03:19
3 Annie Leonhart,04:15
4 Levi Ackerman,10:30
5 Hange Zoe,12:45
6 Armin Arlert,16:01
7 Zeke Yeager,16:43
8 Connie Springer,17:56
9 Erwin Smith,19:18
10 Sasha Braus,23:59
11
```

Testing 2:

Adding three new patient records to the input file which are Ymir,01:01, Bertolt Hoover,06:59 and Marco Bodt,21:49

```
D:\utm_3\dsa\project.exe
Enter input file name: input.txt
Enter output file name: output.txt
Patients sorted and written to output.txt
Press any key to continue . . . |
```

input.txt	
1	Levi Ackerman,10:30
2	Eren Eager,01:40
3	Hange Zoe,12:45
4	Armin Arlert,16:01
5	Annie Leonhart,04:15
6	Erwin Smith,19:18
7	Zeke Yeager,16:43
8	Sasha Braus,23:59
9	Connie Springer,17:56
10	Jean Kirchtein,03:19
11	Ymir,01:01
12	Bertolt Hoover,06:59
13	Marco Bodt,21:49
14	

The output file shows a total of 13 sorted patient records.

output.txt	
1	Ymir,01:01
2	Eren Eager,01:40
3	Jean Kirchtein,03:19
4	Annie Leonhart,04:15
5	Bertolt Hoover,06:59
6	Levi Ackerman,10:30
7	Hange Zoe,12:45
8	Armin Arlert,16:01
9	Zeke Yeager,16:43
10	Connie Springer,17:56
11	Erwin Smith,19:18
12	Marco Bodt,21:49
13	Sasha Braus,23:59
14	

Testing 3:

Deleting two patients from the input file which are Armin Arlert,16:01 and Annie Leonhart,04:15

```
D:\utm_3\dsa\project.exe
Enter input file name: input.txt
Enter output file name: output.txt
Patients sorted and written to output.txt
Press any key to continue . . . |
```

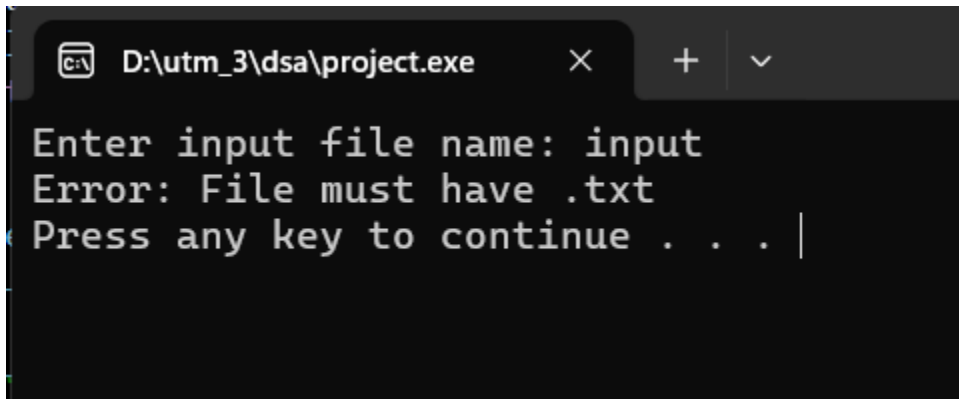
```
input.txt
1  Levi Ackerman,10:30
2  Eren Eager,01:40
3  Hange Zoe,12:45
4  Erwin Smith,19:18
5  Zeke Yeager,16:43
6  Sasha Braus,23:59
7  Connie Springer,17:56
8  Jean Kirchtein,03:19
9  Ymir,01:01
10 Bertolt Hoover,06:59
11 Marco Bodt,21:49
12
```

The output file shows a total of 11 sorted patient records

```
output.txt
1  Ymir,01:01
2  Eren Eager,01:40
3  Jean Kirchtein,03:19
4  Bertolt Hoover,06:59
5  Levi Ackerman,10:30
6  Hange Zoe,12:45
7  Zeke Yeager,16:43
8  Connie Springer,17:56
9  Erwin Smith,19:18
10 Marco Bodt,21:49
11 Sasha Braus,23:59
12
```

Testing 4:

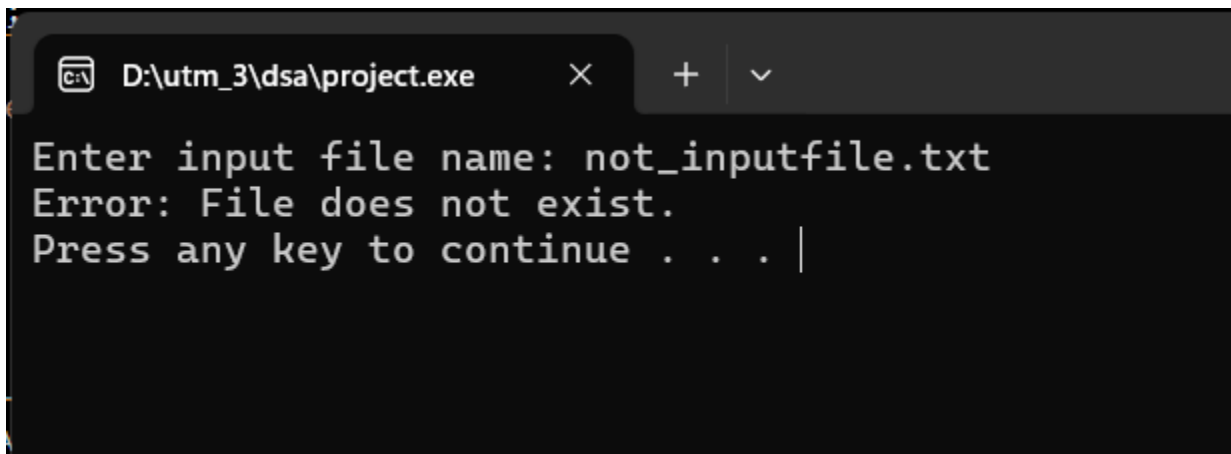
Users entering the file name without the .txt format will pop out an error message.



```
D:\utm_3\dsa\project.exe
Enter input file name: input
Error: File must have .txt
Press any key to continue . . . |
```

Testing 5:

Users entering files that do not exist will pop out an error message.



```
D:\utm_3\dsa\project.exe
Enter input file name: not_inputfile.txt
Error: File does not exist.
Press any key to continue . . . |
```

Testing 6:

Check the 24-hour format and the format of the data in the input file.

Line 12 shows an invalid time format. The time length for the data is too long.

Line 13 shows an invalid time format. The hour and minute should be separated by ':'.

Line 14 shows an invalid time format. The 24-hour format should be in digits.

Line 15 shows an invalid time format. The hour should not be more than 23.

Line 16 shows an invalid time format. The minute entered more than 59.

Line 17 shows that the invalid data format is entered. After the name, it should follow by ', '.

```
input.txt
1 Levi Ackerman,10:30
2 Eren Eager,01:40
3 Hange Zoe,12:45
4 Erwin Smith,19:18
5 Zeke Yeager,16:43
6 Sasha Braus,23:59
7 Connie Springer,17:56
8 Jean Kirchtein,03:19
9 Ymir,01:01
10 Bertolt Hoover,06:59
11 Marco Bodt,21:49
12 NO1,12:345
13 NO2,01.30
14 NO3,a1:40
15 NO4,24:56
16 NO5,01:60
17 NO6:155:45
```

```
output.txt
1 Ymir,01:01
2 Eren Eager,01:40
3 Jean Kirchtein,03:19
4 Bertolt Hoover,06:59
5 Levi Ackerman,10:30
6 Hange Zoe,12:45
7 Zeke Yeager,16:43
8 Connie Springer,17:56
9 Erwin Smith,19:18
10 Marco Bodt,21:49
11 Sasha Braus,23:59
12
```

```
D:\utm_3\dsa\project.exe
Enter input file name: input.txt
Error: Invalid time format at line 12: 12:345
Time should be in 24-hour format (HH:MM)

Error: Invalid time format at line 13: 01.30
Time should be in 24-hour format (HH:MM)

Error: Invalid time format at line 14: a1:40
Time should be in 24-hour format (HH:MM)

Error: Invalid time format at line 15: 24:56
Time should be in 24-hour format (HH:MM)

Error: Invalid time format at line 16: 01:60
Time should be in 24-hour format (HH:MM)

Error: Invalid format at line 17: NO6:155:45

Enter output file name: output.txt
Patients sorted and written to output.txt
Press any key to continue . . . |
```

6.0 Conclusion & Reflection

The project successfully achieved its primary goal of developing a system to manage hospital appointment queues efficiently. By utilizing the Queue data structure and implementing the Sequential Sort algorithm, the system ensures that patient records are sorted and processed in chronological order. The file input/output functionality enhances usability, allowing hospital staff to seamlessly manage appointment records. Testing results validate the system's accuracy and effectiveness for small datasets, providing a reliable solution for reducing appointment scheduling errors and improving operational efficiency.

Reflection

This project offered valuable insights into the practical implementation of data structures and algorithms in solving real-world problems. Through this work, we gained hands-on experience in coding and debugging C++ programs, particularly in areas like dynamic memory allocation and file operations. Additionally, we learned the importance of integrating error-handling mechanisms to ensure robustness.

Challenges encountered included optimizing the sorting process for larger datasets and ensuring memory efficiency with the linked list implementation. These challenges highlighted the need for advanced algorithms and more efficient data structures in future iterations. Overall, this project enhanced our understanding of teamwork, problem-solving, and the application of computational techniques to streamline hospital operations.

7.0 References

- 1) *What is Sorting in C++: Bubble Sort, Insertion Sort & More | Simplilearn.* (n.d).
Simplilearn.com.
https://www.simplilearn.com/tutorials/cpp-tutorial/sorting-in-cpp#types_of_sorting_techniques
- 2) *C++ Program to Implement Queue using Linked List.* (n.d.). Wwww.tutorialspoint.com.
<https://www.tutorialspoint.com/cplusplus-program-to-implement-queue-using-linked-list>